



Universidad Nacional de Rosario

Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Trabajo práctico N° 2

Tecnicatura Universitaria en Inteligencia Artificial

Aprendizaje Automático 2 (IA5.1)

Integrantes	Legajo	Mail
Martinez Dufour, Caterina	M-7169/2	caterinaxmd7@gmail.com
Grimaldi, Damián Daniel	G-5977/3	grimaldidamian001@gmail.com
Tapia, Fabrizio	T-3095/3	fabriziojt1@gmail.com

Fecha de entrega: 05/06/2026

Docentes: Salvaña, Leandro - Fernández, Florencia

ÍNDICE

Problema 1.....	4
1. Introducción y objetivos.....	4
2. Análisis Exploratorio y Preprocesamiento de Datos.....	4
2.1 Descripción del Dataset y Variables.....	4
2.2 Preparación de Datos y Consistencia Dimensional.....	4
3. Estrategia de Carga de Datos y Aumento de Datos (Data Augmentation).....	5
4. Diseño de la Arquitectura del Modelo.....	5
5. Desarrollo y Análisis de Experimentos.....	6
5.1 Experimentos Base y Ajustes Manuales.....	6
5.2 Optimización Hiperparamétrica mediante Búsqueda Bayesiana (Optuna).....	7
6. Evaluación Detallada del Mejor Modelo y Análisis de Resultados.....	7
7. Conclusión.....	8
Problema 2.....	9
1. Introducción y objetivos.....	9
2. Análisis Exploratorio y Preprocesamiento de Datos.....	9
2.1 Descripción del Dataset y contenido del mismo.....	9
3. Diseño e implementación del modelo YOLO.....	11
3.1 Análisis cuantitativo del entrenamiento.....	11
3.1 Comparativa del comportamiento de las curvas.....	11
4. Resultados.....	13
4.1 Análisis del Dashboard de Métricas Generales.....	13
4.2 Análisis de las Matrices de Confusión.....	14
4.3 Análisis de las Curvas de Rendimiento.....	14
4.3 Análisis del Dashboard de Métricas Generales.....	15
4.4 Análisis de las Matrices de Confusión.....	16
4.3 Análisis de las Predicciones en el Conjunto de Test.....	17
5. Conclusiones.....	17
Problema 3.....	19
1. Introducción y objetivos.....	19
2. Análisis Exploratorio y Preprocesamiento de Datos.....	19
2.1 Normalización Temporal de los Audios.....	19

2.2 Generación y Visualización de Espectrogramas de Mel.....	20
2.3 Estandarización del Dataset.....	20
3. Estrategia de Carga de Datos por Lotes (Batching).....	20
4. Diseño de la Arquitectura del Modelo (CRNN).....	21
5. Estrategia Dinámica de Entrenamiento.....	22
6. Desarrollo y Análisis de Experimentos.....	23
6.1 Experimento 1 (E1): Configuración Base con GRU Bidireccional.....	23
6.2 Experimento 2 (E2): Ventana Estrecha y Celda LSTM Bidireccional.....	23
6.1 Experimento 1 (E1): Configuración Base con GRU Bidireccional.....	24
6.2 Experimento 2 (E2): Ventana Estrecha y Celda LSTM Bidireccional.....	25
6.3 Experimento 3 (E3): Ventana Ancha, CNN Profunda y Average Pooling.....	25
6.4 Tabla Comparativa de Experimentos.....	26
7. Evaluación detallada del Mejor Modelo (Matriz de confusión).....	26
8. Conclusión.....	27

Problema 1

1. Introducción y objetivos

En este primer apartado se establece el marco teórico del laboratorio. El objetivo fundamental es construir, entrenar y optimizar una red neuronal convolucional (CNN) desde cero, es decir, prescindiendo por completo de técnicas de aprendizaje por transferencia (transfer learning). El propósito práctico radica en experimentar de forma metodológica y sistemática con diversas elecciones arquitectónicas y configuraciones de hiperparámetros para maximizar la métrica de exactitud (accuracy) en la clasificación de imágenes y mitigar de forma activa el sobreajuste.

2. Análisis Exploratorio y Preprocesamiento de Datos

2.1 Descripción del Dataset y Variables

El problema técnico se define como una clasificación multiclase. Las variables del conjunto de datos son:

- Variables explicativas (features): Imágenes digitales en formato tridimensional RGB con dimensiones y resoluciones espaciales heterogéneas u originales variables.
- Variable objetivo (target): Una etiqueta categórica que diferencia de forma discreta entre 70 razas de perros distintas

2.2 Preparación de Datos y Consistencia Dimensional

Al inicio del pipeline de código se define la función `set_seed(seed=42)`. Su objetivo es fijar las semillas aleatorias de los generadores de Python, NumPy y los backends deterministas de PyTorch y CUDA (`torch.manual_seed`, `deterministic = True`, etc.). Esto

anula la variabilidad estocástica y garantiza la estricta reproducibilidad de las métricas obtenidas en cada iteración. Asimismo, el dataset indexado en el archivo comprimido dataset_perros.zip se descarga mediante gdown y se extrae en el directorio estructurado problema_1/dataset_problema1, registrando un total de 9346 registros anotados validados mediante la lectura del archivo de metadatos dogs.csv.

3. Estrategia de Carga de Datos y Aumento de Datos (Data Augmentation)

Las imágenes extraídas se encuentran segmentadas de forma nativa en subcarpetas de entrenamiento, validación y prueba. Para procesarlas, se utiliza la clase estándar ImageFolder de PyTorch, mapeando la estructura jerárquica de directorios en tensores continuos organizados en minilotes (mini-batches) a través de un objeto DataLoader.

Debido a que el volumen original de datos por clase expone al modelo a memorizar ruido, se implementa un pipeline de transformaciones aleatorias (torchvision.transforms) exclusivo para el set de entrenamiento. Este incluye transformaciones espaciales como volteos horizontales aleatorios (RandomHorizontalFlip), rotaciones angulares y modificaciones de jitter de color (ajustes estocásticos de brillo, contraste y saturación). Con esto se introduce ruido controlado al modelo, impidiendo que los filtros convolucionales se sobreajusten a matrices de píxeles estáticas, forzándolos a extraer patrones geométricos y morfológicos más abstractos e independientes de las condiciones lumínicas.

4. Diseño de la Arquitectura del Modelo

El modelo se diseña como una arquitectura piramidal progresiva constituida por múltiples bloques de convolución bidimensional (Conv2d). Las capas iniciales utilizan kernels pequeños de 3x3 para capturar rasgos primitivos locales de baja abstracción (bordes, esquinas, variaciones cromáticas básicas). A medida que se profundiza en la red, se incrementa el número de canales o mapas de características concurrentes para

codificar abstracciones anatómicas de alto nivel que permitan caracterizar a las razas de perros.

Cada bloque de extracción jerárquica se compone secuencialmente de:

1. Capa Convolutiva (Conv2d): Extractor base de características espaciales.
2. Normalización por Lotes (BatchNormalization): Estabiliza la distribución de las activaciones internas a lo largo del entrenamiento, mitigando el desvanecimiento de gradientes y acelerando de forma drástica la convergencia matemática del optimizador.
3. Activación ReLU: Aplica la rectificación lineal no lineal necesaria para aprender fronteras de decisión complejas.
4. Capa de Max Pooling: Reduce a la mitad las dimensiones espaciales de los mapas de características, induciendo invariancia a pequeñas traslaciones espaciales y reduciendo el coste computacional acumulado.

Al final de los bloques convolucionales se incorpora una capa de agrupamiento adaptativo (AdaptiveAvgPool2d) para aplanar de forma agnóstica la salida espacial y conectarla con la sección de clasificación completamente conectada (Fully Connected Layers o capas lineales densas). Estas capas incorporan regularización mediante Dropout (apagado estocástico de neuronas) para evitar la coadaptación y culminan en una capa lineal final con 70 neuronas de salida correlacionadas con las clases del target.

5. Desarrollo y Análisis de Experimentos

5.1 Experimentos Base y Ajustes Manuales

En las primeras fases experimentales se evalúan y comparan arquitecturas base con diferentes profundidades y tasas de aprendizaje fijas utilizando optimizadores iterativos como SGD y Adam. Se observa una marcada tendencia al overfitting prematuro si las tasas de Dropout o las regularizaciones por decaimiento de pesos (weight decay) son insuficientes, lo cual se constata cuando la función de pérdida del

conjunto de entrenamiento decae continuamente mientras que la pérdida del conjunto de validación diverge hacia valores elevados.

5.2 Optimización Hiperparamétrica mediante Búsqueda Bayesiana (Optuna)

Para automatizar la sintonía fina de los parámetros de entrenamiento, se integra el framework de optimización Optuna. Se define una función objetivo para muestrear de forma inteligente un espacio multidimensional de hiperparámetros que incluye la tasa de aprendizaje logarítmica (learning rate), coeficientes de regularización L2 y el porcentaje óptimo de Dropout. Basado en el algoritmo TPE (Tree-structured Parzen Estimator), el sistema prioriza las regiones más prometedoras de validación, permitiendo consolidar la configuración final óptima del pipeline.

6. Evaluación Detallada del Mejor Modelo y Análisis de Resultados

El modelo definitivo que arrojó el mejor desempeño es el correspondiente al Experimento 4, el cual incorpora de manera conjunta el pipeline de Data Augmentation y una tasa optimizada de Dropout de 0.4 seleccionada por búsqueda bayesiana con Optuna, alcanzando una exactitud final del 54% en el conjunto de validación.

Para diagnosticar de manera rigurosa la capacidad predictiva del sistema, se genera una matriz de confusión multiclase utilizando la librería scikit-learn. El análisis visual y numérico de la matriz confirma que los aciertos se concentran fuertemente sobre la diagonal principal, validando que el clasificador distingue correctamente a la mayoría de las clases. Las confusiones remanentes y marginales se asocian de manera exclusiva a limitaciones biológicas inherentes a la naturaleza de los datos: ambigüedades morfológicas extremas (como las orejas caídas o la longitud del pelaje en ciertos tipos de sabuesos) y similitudes cromáticas globales (patrones de color de pelajes análogos entre razas próximas).

7. Conclusión

El laboratorio demuestra que, ante la ausencia de transferencia de conocimiento preentrenado (Transfer Learning), el equilibrio arquitectónico es crítico para el éxito del entrenamiento. Los resultados del Experimento 4 prueban empíricamente que la combinación de normalización por lotes, inyección de ruido sintético a través de Data Augmentation y el descarte de neuronas por Dropout evitan que la red memorice texturas locales o mapas de píxeles estáticos. Como contrapartida, la regularización directa obliga al modelo convolucional a extraer características invariantes abstractas y patrones anatómicos reales, consolidando un clasificador estable con una robusta capacidad de generalización ante imágenes completamente nuevas del entorno real.

Problema 2

1. Introducción y objetivos

En este segundo apartado se aborda la problemática de detección y clasificación de señales de tráfico mediante técnicas de visión por computadora basadas en aprendizaje profundo. El objetivo principal de este proyecto es aplicar transfer learning sobre un modelo YOLO preentrenado para detectar y clasificar señales de tráfico en imágenes reales. El conjunto de datos utilizado contiene 15 clases diferentes, incluyendo semáforos, señales de límite de velocidad y señales de detención.

El desarrollo del ejercicio abarca el flujo completo de entrenamiento y ajuste fino de un detector de objetos moderno, desde la inspección y validación visual de las anotaciones del dataset hasta el entrenamiento, evaluación y comparación de distintas variantes del modelo. Asimismo, se busca analizar el desempeño obtenido mediante métricas de detección y estudiar la capacidad del modelo para generalizar sobre imágenes no vistas, identificando fortalezas, limitaciones y posibles fuentes de error en las predicciones.

2. Análisis Exploratorio y Preprocesamiento de Datos

2.1 Descripción del Dataset y contenido del mismo.

Para garantizar la reproducibilidad de los experimentos, se utilizó una semilla aleatoria con valor **42**, permitiendo obtener los mismos resultados en futuras ejecuciones bajo las mismas condiciones.

El dataset se encuentra organizado siguiendo el formato utilizado por **Ultralytics YOLO**, con la siguiente estructura:

dataset/

| -- data.yaml

| -- train/

| |-- images/

| | +-- labels/

| -- valid/

| |-- images/

| | +-- labels

+-- test/ 10

 |-- images/

 +-- labels/

El archivo data.yaml contiene la configuración necesaria para el entrenamiento, incluyendo las rutas de acceso a los conjuntos de datos, el número de clases y sus respectivas etiquetas.

Cada archivo de anotaciones con extensión .txt almacena la información correspondiente a los objetos presentes en una imagen. Estas anotaciones se encuentran en el formato requerido por YOLO, donde las coordenadas de los cuadros delimitadores (*bounding boxes*) están normalizadas en el rango de 0 a 1.

El dataset incluye las imágenes destinadas a las etapas de entrenamiento, validación y prueba, junto con sus respectivas anotaciones y el preprocesamiento previamente realizado para su utilización en el modelo.

3. Diseño e implementación del modelo YOLO

Con el objetivo de evaluar el desempeño de YOLO, se decidió utilizar los modelos de yolo v8 tanto de “nano” como de “small”, en la cual, por limitaciones de nuestro hardware, decidimos utilizar la tarjeta gráfica que nos proporciona colab, que nos redujo significativamente el tiempo de entrenamiento.

Tantos en ambos modelos, se utilizó 50 épocas, con un tamaño estándar de imágenes de 640 píxeles,

3.1 Análisis cuantitativo del entrenamiento.

Una vez realizado el entrenamiento de los modelos, se obtuvo la siguiente tabla:

Modelo	Parámetros	mAP@50	mAP@50-95	Tiempo de entrenamiento	Tiempo de Inferencia (por img)
YOLOv8 Nano (yolov8_nano_v1)	3.008.573	0,972	0,836	0,880 horas (~53 minutos)	2,2 ms
YOLOv8 Small (yolov8_small_v1)	11.131.389	0,976	0,844	1,040 horas (~62 min)	4,6 ms

A partir de esta tabla, podemos observar, que el modelo de YOLOv8 Small, tuvo un mejor rendimiento, a pesar de que tardó más en analizar las imágenes, y por consecuencia, de tomarse mayor cantidad de tiempo, pudo obtener un mejor resultado.

3.1 Comparativa del comportamiento de las curvas.

- **Convergencia y Evolución de Pérdidas:**
 - Ambos modelos iniciaron con un comportamiento regular y mostraron un descenso drástico en las curvas de pérdida de caja (`box\_loss`) y clasificación (`cls\_loss`) a partir de la época 41, momento en que el

cargador de datos cierra la técnica de aumentación por mosaico (`Closing dataloader mosaic`).

- Al cerrar el mosaico en la época 50, **YOLOv8 Small** logró minimizar la pérdida de clasificación (`cls\_loss: 0.2622`) en una escala notablemente menor que **YOLOv8 Nano** (`cls\_loss: 0.3248`), demostrando que su mayor volumen de parámetros le permite refinar mejor los rasgos geométricos de las señales de tráfico.
- **Rendimiento por Categoría de Señal (mAP@50-95):**
 - **Señales de luces (Green/Red Light):** Fueron las categorías más desafiantes para ambas redes. Sin embargo, el modelo **Small** pegó un salto de calidad rotundo en “Green Light”, escalando el mAP de **0,882 (nano)** a **0,924 (Small)**. Esto corrobora que las capas adicionales procesan con mayor éxito los objetos texturizados pequeños en entornos viales complejos.
 - **Límites de Velocidad e Indicadores Estructurales:** Las señales numéricas (`Speed Limit 100` a `120`) e imperativas (`Stop`) rozaron la perfección en ambas arquitecturas (métricas sobre 0.985), evidenciando que el dataset provee características lo suficientemente legibles para que incluso una red compacta como la Nano logre generalizar sin dificultades.
- **Conclusión del Trade-off:**
 - El incremento en la complejidad del modelo **Small** (triplica la cantidad de parámetros del Nano) represento solo un **18% más de tiempo de entrenamiento** corriendo sobre la GPU Tesla T4.
 - En términos de inferencia en producción, **YOLOv8 Nano es el doble de rápido (2,2 ms vs 4,6)**. Sin embargo, dado que ambos tiempos se sitúan muy por debajo del umbral estándar para procesamiento en tiempo real (30 FPS $\approx$ 33ms), el incremento en la precisión general (mAP@50-95 de 0,844) y el notable rescate de las clases complejas justifican plenamente la adopción del modelo **Small** para esta aplicación específica.

4. Resultados

4.1 Análisis del Dashboard de Métricas Generales.

1. Evolución de las Perdidas (Losses):

- a. **Pérdida de Caja:** Muestra un descenso constante tanto en entrenamiento como en validación, estabilizando en la validación cerca de 0,57. Esto indica que el modelo aprendió correctamente a ajustar y localizar las bounding boxes alrededor de las señales de tránsito.
- b. **Pérdida de Clasificación:** Exhibe una caída drástica en las primeras 15 épocas. Se observa un escalón de descenso abrupto y definitivo en la **época 41** (momento exacto en que se desactiva la aumentación por mosaico), logrando estabilizar la pérdida de validación en un valor óptimo cercano a 0,30.
- c. **Pérdida de Distribución Focal:** Sigue la misma tendencia descendente y regular, lo que valida que el modelo ganó precisión al refinar los bordes finos de los objetos.

2. Diagnóstico de Overfitting:

- a. Al comparar de forma directa las curvas de train con las de val, se observa que las métricas de validación acompañan en todo momento la caída de las de entrenamiento sin sufrir rebotes o bifurcaciones alcistas hacia las épocas finales. Esto demuestra que **el modelo generaliza correctamente** y que no sufrió problemas de sobreajuste.

3. Métricas de rendimiento General:

- a. Las curvas de **Precisión** y **Recall** trepan rápidamente durante las primeras 10 épocas, estabilizándose por encima del 90% hacia el final del proceso.
- b. Los indicadores de **mAP50** y **mAP50-95** consolidan una curva de crecimiento impecable. El mAP50 se estabiliza de forma muy temprana por encima del 95% mientras que la métrica más exigente continúa refinando gradualmente a lo largo de las 50 épocas hasta consolidar su valor final superior al 83%.

4.2 Análisis de las Matrices de Confusión.

1. Clases con mejor rendimiento:

- Las señales de **Speed Limit 110**, **Speed Limit 70** y **Stop** alcanzan casi la perfección con un 1, 0,99 y 0,99 de efectividad.
- En general, todos los límites de velocidad numéricos se detectan muy bien, lo que demuestra que el modelo diferencia bien los números dentro del círculo rojo.

2. Confusiones con el fondo:

- El error más repetido ocurre cuando el modelo no ve una señal y la confunde con el fondo de la imagen (background).
- Esto afecta principalmente a **Green Light** (un 0,32 se pierde con el fondo) y a **Red Light** (un 0,22 se pierde con el fondo). Al ser objetos más chicos o integrados en estructuras urbanas complejas a la red le cuesta más separarlos del entorno.

3. Confusiones entre clases similares:

- Existe una confusión lógica pero baja entre velocidades parecidas: por ejemplo, un 4% de las señales de **Speed Limit 40** se confunden con **Speed Limit 30**, y un 5% de **Speed Limit 90** se confunde con **Speed Limit 80**. Esto se debe a la similitud gráfica entre los caracteres.
- También hay falsos positivos leves en el fondo, sumando un 14% y 11% respectivamente.

4.3 Análisis de las Curvas de Rendimiento.

1. Curva F1 vs Confianza:

- El punto de equilibrio óptimo se alcanza con un **F1-Score de 0,96 a un umbral de confianza de 0,408**.
- Interpretación práctica:** Al ser una red más robusta, el umbral operativo óptimo sube levemente a 0,41, logrando un rendimiento general superior y más seguro antes de empezar a perder detecciones.

2. Curva Precisión-Recall:

- El modelo considera un **mAP@0.5 general de 0,976**.

- b. La mejora más importante se ve en la clase más difícil, **Green Light**, que sube su mAP individual a 0,924. Esto confirma que las capas extras del modelo Small ayudan a procesar mejor los objetos pequeños y complejos.

3. Curvas de precisión y recall vs confianza:

- a. **Precisión vs Confianza:** Se estabiliza de forma más limpia en la parte superior. Alcanza la precisión perfecta de **1,00 a un umbral de 0,959**. Es un modelo más seguro que comete menos errores a umbrales altos.
- b. **Recall vs Confianza:** Parte con el mismo valor inicial, la curva azul se mantiene “inflada” y plana hacia la derecha. Esto significa que a medida que subimos la exigencia de confianza, el modelo Small es capaz de retener muchas más señales verdaderas antes de que la curva empiece a caer.

4.3 Análisis del Dashboard de Métricas Generales.

1. Optimización de las Pérdidas:

- a. **Pérdida de Caja:** En validación de forma limpia y se estabiliza cerca de 0,55 hacia la época 50. Logra encuadrar las bounding boxes de manera más ajustada que el modelo Nano.
- b. **Pérdida de Clasificación:** Es el punto más fuerte. En validación cae de forma continua y suave hasta tocar un valor óptimo de 0,25. Esto demuestra que el modelo Small tiene menos dudas al etiquetar qué señal es cada cuál.
- c. En la **época 41** se nota el clásico escalón hacia abajo en las pérdidas de entrenamiento, que es cuando el sistema desactiva el mosaico de imágenes para darle el refinamiento final al modelo.

2. Diagnóstico de salud del entrenamiento:

- a. Al igual que el Nano, este entrenamiento es sumamente sano: **no hay ningún síntoma de overfitting (sobreajuste)**. Las líneas de validación (val) bajan a la par de las de entrenamiento (train) en todo momento, confirmando que lo que aprende en el laboratorio lo sabe aplicar bien en datos nuevos.

3. Métricas de Precisión finales:

- a. Las curvas de **mAP50** y **mAP50-95** muestran un crecimiento muy firme y con menos oscilaciones que el modelo Nano.
- b. La métrica más estricta (mAP50-95) se consolida por encima del **84% (0,844)** en validación, coronando un entrenamiento muy estable y preciso

4.4 *Análisis de las Matrices de Confusión.*

1. Efectividad en la Diagonal Principal:

- a. El modelo Small muestra una precisión impresionante: las clases **Speed Limit 100**, **Speed Limit 70** y **Stop** alcanzan puntuaciones casi perfectas de 1,00, 1,00 y 0,99.
- b. Prácticamente todos los límites de velocidad se consolidan por encima del 0.96 de aciertos, lo que demuestra la madurez del modelo para reconocer los caracteres numéricos internos.

2. Mitigación de Errores con las Luces de Tránsito:

- a. La mejora más rotunda respecto al Nano se da en la detección de **Green Light** y **Red Light**.
- b. El acierto de Green Light sube al 0,93 (frente al 0.87 del Nano) y el de Red Light sube al 0,89 (frente al 0,86 del Nano). Las capas extra de la arquitectura Small lograron rescatar gran parte de estas señales que antes se perdían.

3. Confusiones Residuales:

- a. A pesar de las mejoras, el error principal sigue siendo el fondo (`background`). Un 0,37 de las luces verdes y un 0,30 de las luces rojas se siguen ignorando o confundiendo con el entorno urbano, aunque en menor medida que en el modelo Nano.
- b. Las confusiones entre números similares (como confundir un límite de 40 con uno de 30) se redujeron a valores casi insignificantes (apenas un 2%), consolidando un sistema mucho más seguro.

4.3 *Análisis de las Predicciones en el Conjunto de Test*

Contrastamos los resultados visuales de ambos modelos sobre la misma muestra aleatoria de imágenes de test:

- **Rendimiento con Objetos de Gran Tamaño y Definición (Imágenes 1, 2 y 4):** Ambos tamaños de YOLOv8 demuestran un rendimiento sobresaliente y robusto (confianzas superiores al 94%), logrando delimitar las bounding boxes de forma milimétrica. En estos escenarios simples, el uso del modelo **Nano** es altamente eficiente, ya que iguala la precisión de **Small** con un menor costo computacional.
- **El Impacto de la Escala y Distancia (Imagen 3 - Speed Limit 70):** En la detección de la señal lejana y de escala reducida en la ruta, el modelo **Small** supera al Nano alcanzando un 0,82 de confianza frente al 0,79 de este último. Esto justifica el incremento de capas de la arquitectura Small, evidenciando mayor capacidad para retener características de objetos pequeños en capas profundas.
- **Respuesta ante la Degradación de la Imagen (Imagen 5 - Señal Stop Pixelada):** Ante una pérdida severa de resolución (efecto mosaico), el modelo Nano exhibe un score de 0,87, mientras que el Small cae a 0,77. Esto sugiere que el modelo Nano puede presentar una confianza sobreestimada ante patrones ambiguos, mientras que el modelo Small penaliza de forma más realista la pérdida de definición estructural de la señal.

5. Conclusiones

Tras entrenar y evaluar las arquitecturas `YOLOv8 Nano` y `YOLOv8 Small` durante 50 épocas con imágenes de 640x640 píxeles, se presentan las siguientes conclusiones finales:

- **Superioridad Métrica:** El modelo **YOLOv8 Small** superó al Nano alcanzando un **mAP@50-95 de 0,844** frente al **0,836** de este último. Esta ganancia se tradujo visualmente en una mayor seguridad al detectar señales lejanas o pequeñas y

en un aumento drástico del acierto en las luces de tránsito (subiendo de 0,88 a 0.92 en luz verde).

- **Análisis del Trade-off:** Aunque el modelo **Small** triplica la cantidad de parámetros y duplica el tiempo de inferencia por imagen (4.6 ms frente a 2.2 ms del **Nano**), ambos se sitúan muy por debajo del umbral crítico para procesamiento en tiempo real (~33 ms). Por ende, el costo computacional extra está plenamente justificado en pos de la seguridad vial.
- **Calidad del Dataset y Estabilidad:** Ambos entrenamientos resultaron sumamente sanos, sin ningún síntoma de sobreajuste (overfitting). El cierre de la aumentación por mosaico en la época 41 fue clave para estabilizar las pérdidas y darle el refinamiento geométrico final a las bounding boxes.
- **Configuración Óptima en Producción:** Basándose en las curvas de balance F1, se determina que el modelo **Small** debería operar con un umbral de confianza de 0,41 para garantizar el máximo equilibrio entre precisión y recall en un entorno real.

Problema 3

1. Introducción y objetivos

El objetivo principal de este proyecto es diseñar, implementar y optimizar una red neuronal convolucional recurrente (CRNN) capaz de clasificar con alta precisión un conjunto de datos compuesto por 10 comandos de voz de un segundo de duración (extraídos del dataset Mini Speech Commands de TensorFlow). El pipeline completo abarca desde la ingesta de las ondas de audio sin procesar hasta la evaluación fina del modelo definitivo, buscando mitigar el sobreajuste y maximizar la capacidad de generalización en datos no observados (test set).

2. Análisis Exploratorio y Preprocesamiento de Datos

2.1 Normalización Temporal de los Audios

Como los archivos de audio en el disco real pueden presentar variaciones milimétricas en su longitud, es indispensable asegurar la consistencia dimensional antes de alimentar la red neuronal. Para ello, se desarrolló una rutina de normalización que fuerza a todas las ondas a medir exactamente un segundo de duración a una tasa de muestreo (sampling rate) fija de 16,000 Hz (lo que equivale a 16,000 muestras por archivo).

La lógica implementada realiza lo siguiente:

- Si el audio es más largo de un segundo, se recorta de forma exacta quedándose con las primeras 16,000 muestras.
- Si el audio es más corto, se le aplica un relleno de ceros (zero-padding) al final de la señal hasta completar la longitud requerida.

2.2 Generación y Visualización de Espectrogramas de Mel

A partir de la onda normalizada en el tiempo, se genera un Espectrograma de Mel en escala logarítmica (dB) utilizando la librería Librosa. Este proceso transforma la señal acústica en una representación visual densa que organiza la información en dos ejes fundamentales:

- Frecuencia en el eje 0 (Filas): Representa las 64 bandas de frecuencia Mel (la altura de la matriz).
- Tiempo en el eje 1 (Columnas): Representa los pasos temporales (frames) a lo largo de ese segundo de duración (el ancho de la matriz).

Para evaluar la variabilidad acústica, se realizó un gráfico de control con 8 espectrogramas, contrastando 4 instancias pertenecientes a una misma clase ('down') frente a 4 instancias correspondientes a una clase distinta ('up'). En las imágenes se observa claramente cómo la distribución de la energía acústica (escala de color magma) varía sutilmente entre hablantes de un mismo comando, pero cambia de forma drástica al comparar palabras diferentes, proporcionando las características visuales óptimas para el entrenamiento convolucional.

2.3 Estandarización del Dataset

Para estabilizar el gradiente y acelerar la convergencia del optimizador, se calculó la media y el desvío estándar globales a partir de todo el conjunto de entrenamiento, obteniendo un valor de media de -57.21 y un desvío estándar de 19.83. Estos coeficientes fijos se utilizan para estandarizar cada espectrograma antes de ingresarlo al modelo, restando la media y dividiendo por el desvío.

3. Estrategia de Carga de Datos por Lotes (Batching)

Las rutas de archivos de audio de las listas (train_files, val_files, test_files) representan texto en disco y no pueden ser consumidas directamente por la red neuronal. Asimismo, procesar los miles de audios en un único paso saturaría la memoria de la

placa de video (VRAM). Por lo tanto, se diseñó la función `obtener_batch` como la directora de orquesta del pipeline de datos.

Esta rutina toma un lote pequeño (subgrupo de 32 o 64 rutas de archivos), ejecuta en paralelo la carga del audio, la conversión a espectrograma Mel, la extracción mediante ventana deslizante y mapea las etiquetas textuales a números utilizando un diccionario de índices (`label_to_idx`). Al finalizar el ciclo, empaqueta los arreglos en tensores de PyTorch listos para el entrenamiento: un tensor de características X con dimensiones (`Batch_Size`, N , F , W) y un tensor de etiquetas numéricas Y . Para mantener la consistencia matemática, la función descarta los subgrupos residuales finales que queden incompletos.

4. Diseño de la Arquitectura del Modelo (CRNN)

El modelo sigue una arquitectura híbrida CRNN (Convolutional Recurrent Neural Network). Con el objetivo de optimizar los tiempos de cómputo, el método `forward` colapsa inicialmente las dimensiones del lote y de las ventanas secuenciales usando `.view()`. Esto permite que el codificador convolucional genérico (`CNNEncoderGenerico`) procese todas las ventanas de forma paralela simultáneamente, extrayendo las características visuales fundamentales de los espectrogramas.

Tras la extracción convolucional, el flujo de datos reestructura la secuencia temporal original mediante la operación `.view()`, adaptando las dimensiones al formato tridimensional requerido por la capa recurrente (`batch`, `secuencia`, `embedding`). Esta capa recurrente está diseñada de manera dinámica, permitiendo seleccionar entre celdas LSTM o GRU, operando de forma unidireccional o bidireccional para capturar el contexto total de la palabra. Finalmente, para efectuar la clasificación de las 10 clases, se extrae el estado correspondiente al último paso temporal de la secuencia (o se aplica `average pooling`) y se proyecta a través de una capa lineal final (`fully connected`).

4.1 Configuración Agnóstica al Hardware (CPU vs. GPU)

Para garantizar la portabilidad del código, se configuró la selección dinámica del dispositivo de cómputo. Esto automatiza la detección de entornos compatibles con CUDA de NVIDIA para acelerar drásticamente los cálculos de Deep Learning por hardware. En caso de no contar con una GPU dedicada, el sistema conmuta automáticamente al procesador tradicional (CPU), permitiendo la ejecución estable del script en cualquier computadora sin modificaciones manuales.

5. Estrategia Dinámica de Entrenamiento

El bucle de entrenamiento manual (`loop_entrenamiento_crnn_manual`) recibe la configuración parametrizada de cada experimento y almacena cronológicamente en un historial las pérdidas (`loss`) y precisiones (`accuracy`) de entrenamiento y validación. En cada época, se realiza un barajado aleatorio (`shuffle`) de las rutas para asegurar la robustez estadística. Posterior al cálculo de la pérdida por entropía cruzada, se ejecuta un recorte de gradientes mediante `clip_grad_norm_` (fijado en una norma máxima de 1.0) para estabilizar las capas recurrentes contra el problema de gradientes explosivos.

Con el propósito de mantener limpia la terminal, el progreso del entrenamiento se reporta en consola de forma controlada cada 5 épocas, o bien en cada iteración donde el modelo detecte una mejora en el `validation loss`. Adicionalmente, se incorporó un mecanismo de parada temprana (`early stopping`) acoplado a un umbral de paciencia: si la pérdida de validación no mejora tras una cantidad determinada de épocas consecutivas, la ejecución finaliza de inmediato, previniendo el sobreajuste y restaurando los mejores pesos registrados en el disco duro.

6. Desarrollo y Análisis de Experimentos

6.1 Experimento 1 (E1): Configuración Base con GRU Bidireccional

Este experimento actúa como nuestra línea de base (baseline). Se configuró una ventana deslizante de 16 frames con un solapamiento (hop) de 8. La CNN se compuso de dos bloques convolucionales progresivos [(1, 32), (32, 64)] con un embedding de 128. La RNN utilizó 2 capas de celdas GRU Bidireccionales con 128 unidades ocultas y un dropout intercapa de 0.3. El entrenamiento se inició con una tasa de aprendizaje de 0.001 y el optimizador Adam.

Análisis de Resultados (E1): El experimento 1 funcionó exitosamente demostrando que en cada época se reduce el train loss y aumenta el accuracy. Obtuvo una precisión en test muy elevada del 95.30%, lo cual nos dice que el modelo aprendió correctamente a generalizar con datos nuevos. Al observar su gráfico de pérdida, se nota que en train va descendiendo gradualmente mientras que en accuracy va mejorando su valor en cada ejecución. Si bien val loss y val acc tienen unos pequeños picos debido a las oscilaciones normales del aprendizaje, en ambos gráficos siempre van tendiendo al mejor resultado. Un detalle importante es el quiebre que se produce a partir de la época 12, donde la curva de entrenamiento se acelera y la de validación encuentra su mejor punto; esto coincide con la activación de nuestro scheduler (ReduceLROnPlateau) que redujo la tasa de aprendizaje a la mitad, logrando ajustar el modelo de forma fina y evitando el sobreajuste.

6.2 Experimento 2 (E2): Ventana Estrecha y Celda LSTM Bidireccional

Para esta prueba, se redujo el tamaño de la ventana deslizante a la mitad (ventana de 8 frames con hop de 4), generando una secuencia temporal más larga y detallada. Se preservó el diseño de la CNN [(1, 16), (16, 32), (32, 64)] y se cambió el núcleo recurrente por celdas LSTM Bidireccionales de 2 capas con 128 unidades ocultas, incrementando el dropout a 0.4 para compensar la longitud de la secuencia.

Análisis de Resultados (E2): Para el experimento 2, podemos observar que por época siempre se fue reduciendo tanto el valor de la pérdida (loss) como aumentando el accuracy de entrenamiento. Este modelo obtuvo una precisión en test del 94.88%; si bien es ligeramente menor a la del experimento 1, sigue siendo un valor sumamente alto que demuestra un rendimiento excelente. Al observar las gráficas, notamos un comportamiento muy parejo y similar al primer experimento, donde la pérdida va decreciendo progresivamente y el accuracy aumenta en cada época de forma constante. Un dato interesante a destacar es que, a diferencia del caso anterior, el optimizador no requirió reducir la tasa de aprendizaje (learning rate), manteniéndose estable en $1.00e-03$ durante las 15 épocas completas. Los pequeños picos u oscilaciones de las curvas de validación (como el de la época 9) se corrigieron rápidamente por sí solos en las épocas siguientes. Con esto concluimos que el cambio a celdas LSTM junto con una ventana más estrecha de 8 frames es una alternativa muy sólida y eficiente que también logra evitar por completo el sobreajuste.

6.1 Experimento 1 (E1): Configuración Base con GRU Bidireccional

Este experimento actúa como nuestra línea de base (baseline). Se configuró una ventana deslizante de 16 frames con un solapamiento (hop) de 8. La CNN se compuso de dos bloques convolucionales progresivos [(1, 32), (32, 64)] con un embedding de 128. La RNN utilizó 2 capas de celdas GRU Bidireccionales con 128 unidades ocultas y un dropout intercapa de 0.3. El entrenamiento se inició con una tasa de aprendizaje de 0.001 y el optimizador Adam.

Análisis de Resultados (E1): El experimento 1 funcionó exitosamente demostrando que en cada época se reduce el train loss y aumenta el accuracy. Obtuvo una precisión en test muy elevada del 95.30%, lo cual nos dice que el modelo aprendió correctamente a generalizar con datos nuevos. Al observar su gráfico de pérdida, se nota que en train va descendiendo gradualmente mientras que en accuracy va mejorando su valor en cada ejecución. Si bien val loss y val acc tienen unos pequeños picos debido a las oscilaciones normales del aprendizaje, en ambos gráficos siempre van tendiendo al mejor resultado. Un detalle importante es el quiebre que se produce a partir de la época 12, donde la curva de entrenamiento se acelera y la de validación encuentra su mejor punto; esto coincide con la activación de nuestro scheduler

(ReduceLROnPlateau) que redujo la tasa de aprendizaje a la mitad, logrando ajustar el modelo de forma fina y evitando el sobreajuste.

6.2 Experimento 2 (E2): Ventana Estrecha y Celda LSTM Bidireccional

Para esta prueba, se redujo el tamaño de la ventana deslizante a la mitad (ventana de 8 frames con hop de 4), generando una secuencia temporal más larga y detallada. Se preservó el diseño de la CNN [(1, 16), (16, 32), (32, 64)] y se cambió el núcleo recurrente por celdas LSTM Bidireccionales de 2 capas con 128 unidades ocultas, incrementando el dropout a 0.4 para compensar la longitud de la secuencia.

Análisis de Resultados (E2): Para el experimento 2, podemos observar que por época siempre se fue reduciendo tanto el valor de la pérdida (loss) como aumentando el accuracy de entrenamiento. Este modelo obtuvo una precisión en test del 94.88%. Si bien es ligeramente menor a la del experimento 1, sigue siendo un valor sumamente alto que demuestra un rendimiento excelente. Al observar las gráficas, notamos un comportamiento muy parejo y similar al primer experimento, donde la pérdida va decreciendo progresivamente y el accuracy aumenta en cada época de forma constante. Un dato interesante a destacar es que, a diferencia del caso anterior, el optimizador no requirió reducir la tasa de aprendizaje (learning rate), manteniéndose estable durante las 15 épocas completas. Los pequeños picos u oscilaciones de las curvas de validación (como el de la época 9) se corrigieron rápidamente por sí solos en las épocas siguientes. Con esto concluimos que el cambio a celdas LSTM junto con una ventana más estrecha de 8 frames es una alternativa muy sólida y eficiente que también logra evitar por completo el sobreajuste.

6.3 Experimento 3 (E3): Ventana Ancha, CNN Profunda y Average Pooling

Buscando explorar el extremo opuesto, el experimento 3 ensanchó la ventana deslizante al doble (32 frames con hop de 16) para dar mayor contexto local a la CNN y acortar la secuencia temporal. La estructura convolucional se robusteció duplicando sus canales [(1, 64), (64, 128)] y el embedding se elevó a 256. Se retornó a celdas GRU

Bidireccionales (2 capas con 256 unidades ocultas) pero se removió por completo el regularizador dejando el dropout en 0.0. La tasa de aprendizaje inicial se fijó en un valor más conservador de 0.0001.

Análisis de Resultados (E3): Para el experimento 3, podemos observar que por época siempre se fue reduciendo tanto el valor de la pérdida como aumentando el accuracy de entrenamiento. Este modelo obtuvo una precisión en test del 92.94%; si bien es menor a la de los experimentos anteriores, sigue representando un buen rendimiento para la clasificación. Al observar las gráficas, notamos que en train la pérdida va decreciendo progresivamente y el accuracy aumenta en cada época de forma constante. Sin embargo, las curvas de validación presentan oscilaciones y picos más marcados que en los casos previos. Esto se debe a que, al remover el dropout (0.0), el modelo se vuelve más sensible a las variaciones del conjunto de validación. Un dato muy importante a destacar es que el entrenamiento se detuvo automáticamente en la época 11 debido a la función de parada temprana (early stopping), ya que el mejor valor de val loss se alcanzó en la época 6 (0.2351) y no volvió a mejorar tras cumplirse la paciencia de 5 épocas. Con esto concluimos que, aunque la estrategia de Average Pooling y ventanas anchas es una propuesta interesante, la falta de regularización hizo que el modelo tienda levemente al sobreajuste en las últimas épocas, fijando un límite en su capacidad de generalización.

6.4 Tabla Comparativa de Experimentos

A partir de las métricas consolidadas en la tabla comparativa, se concluye con solidez que el Experimento 1 (E1) es el ganador indiscutido por registrar el mayor accuracy en prueba (95.30%), procediendo a utilizar su arquitectura para el análisis de errores finos.

7. Evaluación detallada del Mejor Modelo (Matriz de confusión)

Al analizar detalladamente la matriz de confusión generada sobre el conjunto de test con el modelo E1, se observa que la red predice correctamente casi todos los casos

para los 10 comandos de voz, acumulando la densidad visual sobre la diagonal principal. No obstante, se logran identificar patrones de confusión específicos muy interesantes:

- no vs. go: En 16 casos el comando real 'no' fue clasificado erróneamente como 'go' (y en 8 ocasiones al revés). Esto halla su causa en que son palabras monosílabas fonéticamente muy parecidas en su estructura de pronunciación y envolvente espectral.
- on vs. off: Se registraron 18 confusiones donde la palabra real 'on' fue interpretada como 'off'. Al ser términos opuestos de longitud similar que comparten contextos idénticos, sus dinámicas espectro-temporales son sumamente cercanas.
- off vs. up: Se evidenciaron 13 casos de error de 'off' clasificados como 'up', denotando una pequeña ambigüedad acústica en la transición de las vocales iniciales abiertas de ambos comandos.
- down vs. go: Registró una confusión menor con 14 casos reportados. Fuera de estos escenarios puntuales, las desviaciones se presentan en una escala marginal. Esto nos permite concluir que los pocos fallos del modelo no responden a debilidades de la red, sino a la similitud fonética intrínseca de las palabras que componen el dataset.

8. Conclusión

En este trabajo práctico aprendimos a implementar un análisis profundo sobre archivos de audio, visualizando sus espectrogramas Mel y aplicando técnicas de normalización temporal, como el recorte y el padding, para garantizar que todas las muestras midieran exactamente un segundo de duración

A lo largo de la experimentación con los 10 comandos de voz, pudimos observar que, si bien algunos términos presentaban una tendencia natural a confundirse debido a sus similitudes fonéticas, el diseño de una estructura robusta (como la arquitectura CRNN

optimizada) permite lograr un entrenamiento altamente eficiente, alcanzando predicciones correctas en la gran mayoría de los casos.

El desarrollo de este tipo de modelos resulta sumamente útil en el ámbito del reconocimiento de voz (Speech Recognition) y el procesamiento de lenguaje natural. Su implementación es clave para el diseño de asistentes virtuales inteligentes, sistemas de control por comandos de voz en dispositivos de hardware o interfaces de accesibilidad, demostrando el enorme potencial de combinar redes convolucionales y recurrentes para interpretar información acústica